La empresa **GameWorld** necesita desarrollar una tienda online de videojuegos.

La aplicación permitirá consultar videojuegos, registrarse como cliente, iniciar sesión, gestionar un carrito de compra y realizar pedidos.

Además, los administradores podrán añadir, modificar y eliminar videojuegos del catálogo.

Para desarrollar la aplicación se utilizarán **PHP, MySQL y MySQLi**, aplicando los conocimientos trabajados en los ejercicios anteriores.

Se proporcionarán la base de datos, el diseño CSS, los formularios HTML y parte del código PHP. El alumnado deberá completar los bloques señalados mediante comentarios `TODO` y crear las funciones necesarias.

Base de datos

Importa el archivo `gameworld.sql` utilizando phpMyAdmin.

La base de datos contiene cuatro tablas:

Tabla	Información almacenada
`usuarios`	Datos de los clientes y administradores
`videojuegos`	Catálogo de productos, precios y stock
`pedidos`	Pedidos realizados por los usuarios
`detalle_pedido`	Videojuegos y cantidades de cada pedido

La importación incluye diez videojuegos y un usuario administrador.

Credenciales del administrador:

- Correo electrónico: `admin@gameworld.local`
- Contraseña: `Admin1234!`

Atención: Importar nuevamente el SQL eliminará los datos anteriores de estas tablas. Las credenciales facilitadas son exclusivamente para las pruebas del aula.

1. Organización del código y funciones reutilizables

Objetivo: Organizar correctamente la aplicación y evitar la repetición de código.

Tareas

1. Crear el archivo `includes/funciones.php`, donde se definirán las funciones generales que no realizan consultas a la base de datos.
2. Crear el archivo `includes/funciones_bd.php`, donde se definirán todas las funciones que ejecutan consultas SQL.
3. Utilizar `includes/conexion.php` para establecer la conexión con MySQL.
4. Utilizar `require_once` para cargar los archivos necesarios.
5. Reutilizar `header.php` y `footer.php` en todas las páginas que generan HTML.
6. Evitar repetir validaciones y operaciones que puedan implementarse mediante funciones.

Funciones que deberán desarrollarse

En `funciones.php`:

- Validación de precios y cantidades.
- Cálculo de subtotales y totales.
- Comprobación de sesiones y permisos.
- Otras operaciones generales que se repitan.

En `funciones_bd.php`:

- Consulta de videojuegos.
- Búsqueda y filtrado.
- Consulta e inserción de usuarios.
- Inserción, modificación y eliminación de videojuegos.
- Registro y consulta de pedidos.
- Otras operaciones necesarias con MySQL.

Resultado esperado: Las páginas de la aplicación llamarán a funciones reutilizables y no contendrán consultas SQL directamente.

2. Registro e inicio de sesión

Objetivo: Permitir que los clientes creen una cuenta y accedan a la tienda mediante una sesión.

2.1. Registro de usuarios

Completa el archivo `registro.php`.

El formulario solicita:

- Nombre.
- Correo electrónico.
- Contraseña.
- Repetición de contraseña.

Al recibir el formulario mediante POST, PHP deberá:

1. Recuperar los datos introducidos.
2. Comprobar que los campos obligatorios no estén vacíos.
3. Validar el formato del correo electrónico.
4. Comprobar que la contraseña tenga al menos ocho caracteres.
5. Comprobar que las dos contraseñas coincidan.
6. Consultar si el correo electrónico ya está registrado.
7. Generar un hash de la contraseña mediante `password_hash()`.
8. Insertar el usuario en la base de datos con el rol `cliente`.
9. Redirigir a `login.php` si el registro se realiza correctamente.

No se permitirá que un usuario seleccione su propio rol durante el registro.

2.2. Inicio de sesión

Completa `login.php`.

Al enviar el correo y la contraseña:

1. Buscar el usuario en la base de datos mediante su correo electrónico.
2. Comprobar la contraseña utilizando `password_verify()`.
3. Si las credenciales son correctas, regenerar el identificador de sesión.
4. Guardar en `$_SESSION` el identificador, el nombre y el rol del usuario.
5. Redirigir a la página principal.
6. Si las credenciales son incorrectas, mostrar un mensaje de error.

2.3. Cierre de sesión

Completa `logout.php`.

Al cerrar sesión se deberán eliminar los datos de sesión, invalidar la cookie de sesión y redirigir al usuario a `index.php`.

2.4. Menú de navegación

Modifica `includes/header.php` para que muestre los enlaces correspondientes:

- Usuario no identificado: Registro y Entrar.
- Usuario identificado: Salir.
- Administrador identificado: también Administración.

Resultado esperado: Un cliente puede registrarse, iniciar sesión y cerrar sesión. El administrador proporcionado puede iniciar sesión con sus credenciales iniciales.

3. Catálogo y búsqueda de videojuegos

Objetivo: Consultar los videojuegos almacenados en MySQL mediante búsquedas, filtros y paginación.

3.1. Mostrar el catálogo

Completa `catalogo.php` para recuperar los videojuegos de la base de datos y mostrar:

- Nombre.
- Género.
- Plataforma.
- Precio.
- Stock.
- Enlace para consultar los detalles.

Si no hay videojuegos que cumplan los criterios de búsqueda, se mostrará un mensaje informativo.

3.2. Buscar y filtrar

El usuario podrá:

1. Buscar videojuegos por una parte de su nombre.
2. Filtrar por plataforma: PC, PS5, Xbox o Switch.
3. Ordenar por nombre.
4. Ordenar por precio ascendente.
5. Ordenar por precio descendente.

Los filtros deberán funcionar conjuntamente.

Por ejemplo, se podrán buscar videojuegos que contengan la palabra «Mario», de la plataforma Switch, ordenados por precio.

Los valores recibidos mediante GET deberán validarse antes de construir la consulta SQL. No se permitirá introducir directamente un valor de ordenación recibido del usuario en la consulta.

3.3. Paginación

El catálogo mostrará **cinco videojuegos por página**.

PHP deberá:

1. Recuperar el número de página mediante `GET`.
2. Comprobar que sea válido.
3. Calcular qué registros deben recuperarse.
4. Consultar el total de videojuegos que cumplen los filtros.
5. Calcular el número total de páginas.
6. Mostrar enlaces para navegar entre páginas.

Al cambiar de página deberán mantenerse los filtros y la ordenación seleccionados.

3.4. Detalles de un videojuego

Completa `detalle.php`.

El identificador del videojuego se recibirá mediante `GET`.

PHP deberá validar el identificador, consultar el videojuego y mostrar todos sus datos.

Si el identificador no es válido o el videojuego no existe, se mostrará un mensaje de error.

Resultado esperado: El catálogo permitirá buscar, filtrar, ordenar y navegar entre páginas sin perder los criterios seleccionados.

4. Carrito de compra

Objetivo: Gestionar un carrito de compra utilizando sesiones.

El carrito se almacenará en `$_SESSION["carrito"]`, guardando el identificador de cada videojuego y la cantidad seleccionada.

Los precios y nombres se recuperarán siempre de MySQL.

4.1. Añadir videojuegos

Desde `detalle.php`, el usuario podrá seleccionar una cantidad y añadir el videojuego al carrito.

Al recibir el formulario, PHP deberá:

1. Comprobar que el videojuego existe.
2. Validar que la cantidad sea un entero positivo.
3. Consultar el stock disponible.
4. Comprobar que la cantidad solicitada no supera el stock.

5. Añadir el videojuego al carrito.

Si el videojuego ya está en el carrito, se incrementará su cantidad sin crear otra entrada.

4.2. Mostrar el carrito

Completa `carrito.php` para mostrar:

- Nombre del videojuego.
- Precio unitario.
- Cantidad seleccionada.
- Subtotal.
- Importe total del carrito.

El subtotal se calculará multiplicando el precio unitario por la cantidad.

El importe total será la suma de todos los subtotales.

4.3. Modificar cantidades

El usuario podrá cambiar la cantidad de un videojuego.

PHP deberá comprobar que la nueva cantidad sea válida y que no supere el stock disponible.

4.4. Eliminar videojuegos

El usuario podrá eliminar un videojuego del carrito.

Esta operación únicamente modificará la sesión y no eliminará el videojuego de la base de datos.

4.5. Carrito vacío

Si el carrito no contiene videojuegos, se mostrará un mensaje informativo y no se permitirá finalizar una compra.

Resultado esperado: El usuario podrá añadir, consultar, modificar y eliminar productos del carrito, con los importes correctamente calculados.

5. Confirmación de pedidos

Objetivo: Registrar compras en la base de datos y actualizar el stock.

5.1. Comprobaciones previas

Antes de registrar un pedido, PHP deberá:

1. Comprobar que el usuario ha iniciado sesión.
2. Comprobar que el carrito no está vacío.
3. Validar los identificadores y cantidades.
4. Recuperar los precios actuales desde MySQL.
5. Comprobar nuevamente el stock disponible.

No se utilizarán precios enviados mediante formularios.

5.2. Registrar el pedido

Desarrolla una función `registrarPedido()` en `funciones_bd.php`.

La función deberá:

1. Iniciar una transacción.
2. Consultar y bloquear los registros de los videojuegos que se van a comprar.
3. Comprobar el stock disponible.
4. Calcular el importe total.
5. Insertar un registro en `pedidos`.
6. Insertar las líneas correspondientes en `detalle_pedido`.
7. Descontar las cantidades compradas del stock.
8. Confirmar la transacción si todas las operaciones son correctas.

Si se produce un error, se deberá ejecutar `ROLLBACK` para deshacer los cambios.

5.3. Finalizar la compra

Si el pedido se registra correctamente:

- Vaciar el carrito de la sesión.
- Mostrar un mensaje de confirmación.
- Permitir consultar el pedido desde el historial.

Si se produce un error:

- Mostrar un mensaje explicativo.
- Mantener el carrito.
- Evitar que se guarde un pedido incompleto.

Resultado esperado: Una compra correcta genera un pedido con sus líneas y descuenta el stock. Una compra incorrecta no produce modificaciones parciales en la base de datos.

6. Historial de pedidos

Objetivo: Permitir que cada cliente consulte sus compras anteriores.

Tareas

1. Comprobar que el usuario ha iniciado sesión.
2. Recuperar únicamente los pedidos correspondientes a su identificador.
3. Mostrar el número de pedido, la fecha y el importe total.
4. Consultar los videojuegos incluidos en cada pedido.
5. Mostrar para cada línea el nombre, la cantidad, el precio unitario y el subtotal.
6. Mostrar un mensaje si el usuario todavía no ha realizado compras.

Un usuario no podrá consultar los pedidos de otro modificando parámetros de la URL.

Resultado esperado: Cada cliente podrá consultar exclusivamente sus propios pedidos y los productos comprados.

7. Administración del catálogo

Objetivo: Permitir que los administradores gestionen los videojuegos.

7.1. Control de acceso

Antes de mostrar cualquier página de administración, PHP deberá comprobar que:

1. El usuario ha iniciado sesión.
2. Su rol es `admin`.

Si no cumple estas condiciones, no podrá acceder a la funcionalidad de administración.

No será suficiente con ocultar los enlaces del menú.

7.2. Listado de videojuegos

Completa `admin/videojuegos.php` para mostrar todos los videojuegos y sus datos principales.

Cada videojuego tendrá enlaces para editarlo y eliminarlo.

También existirá un enlace para añadir nuevos videojuegos.

7.3. Añadir videojuegos

Completa `admin/anadir.php`.

El formulario permitirá introducir:

- Nombre.
- Género.
- Plataforma.
- Precio.

- Stock.

PHP deberá validar los datos y comprobar que no exista otro videojuego con el mismo nombre y plataforma.

Si los datos son correctos, se insertará el videojuego mediante una sentencia preparada `INSERT`.

7.4. Editar videojuegos

Completa `admin/editar.php`.

Al seleccionar un videojuego:

1. Recuperar su identificador mediante `GET`.
2. Validar el identificador.
3. Consultar sus datos actuales.
4. Mostrar los valores en el formulario.
5. Recibir los cambios mediante `POST`.
6. Validar los nuevos datos.
7. Comprobar que los cambios no generan un duplicado de nombre y plataforma.
8. Actualizar el registro mediante `UPDATE`.

7.5. Eliminar videojuegos

Completa `admin/eliminar.php`.

Antes de eliminar un videojuego, se mostrará una página de confirmación.

La eliminación deberá realizarse mediante `POST` y una sentencia preparada `DELETE`.

Si el videojuego aparece en un pedido anterior, no se permitirá eliminarlo físicamente, para conservar el historial de compras.

7.6. Redirecciones

Después de añadir, editar o eliminar correctamente un videojuego, se redirigirá al listado de administración.

Se mostrará un mensaje que indique el resultado de la operación.

Resultado esperado: El administrador podrá gestionar el catálogo. Los clientes normales no podrán acceder a estas operaciones.

8. Preferencias mediante cookies

Objetivo: Guardar las preferencias visuales del usuario.

Archivos: `preferencias.php` e `includes/header.php`.

Tareas

1. Completar el formulario de selección del tema visual.
2. Permitir elegir entre `claro` y `oscuro`.
3. Recuperar el valor mediante `POST`.
4. Comprobar que el tema recibido es válido.
5. Guardar la preferencia en una cookie con una duración de siete días.
6. Recuperar la cookie desde `header.php`.
7. Aplicar el tema seleccionado a todas las páginas de la aplicación.

Si la cookie no existe o contiene un valor no válido, se utilizará el tema claro.

La preferencia deberá mantenerse aunque el usuario cierre la sesión.

Resultado esperado: El usuario podrá cambiar el tema visual y conservar su elección al volver a abrir la aplicación.

9. Validación, seguridad y comprobaciones finales

Objetivo: Revisar el funcionamiento de toda la aplicación y evitar operaciones incorrectas.

9.1. Validación de datos

Todos los datos recibidos mediante `GET` y `POST` deberán validarse en PHP.

Se comprobarán, entre otros:

- Identificadores enteros positivos.
- Cantidades enteras válidas.
- Precios mayores que cero.
- Campos obligatorios.
- Correos electrónicos.
- Plataformas permitidas.
- Identificadores inexistentes.

9.2. Consultas y presentación

Todas las consultas con datos externos deberán utilizar sentencias preparadas.

Los textos mostrados en HTML deberán escaparse mediante `htmlspecialchars()` cuando corresponda.

9.3. Protección de formularios

Los formularios que modifiquen información deberán incorporar un token CSRF.

El token se almacenará en la sesión y se comprobará antes de realizar la operación.

9.4. Redirecciones

Después de procesar correctamente formularios que modifican datos, se utilizará el patrón **POST-Redirect-GET**.

Las redirecciones y la creación de cookies deberán realizarse antes de generar contenido HTML.

9.5. Pruebas obligatorias

Realiza las siguientes comprobaciones:

Prueba	Resultado esperado
Registrar un cliente válido	El usuario se guarda en MySQL
Registrar un correo repetido	Se rechaza el registro
Introducir una contraseña incorrecta	No se inicia sesión
Iniciar sesión como administrador	Se permite acceder a administración
Acceder a administración como cliente	Acceso denegado
Buscar videojuegos por nombre y plataforma	Se muestran únicamente los resultados correspondientes
Cambiar de página en el catálogo	Se mantienen los filtros
Añadir dos veces el mismo videojuego	Se acumulan las cantidades
Solicitar más unidades que el stock	Se rechaza la operación
Comprar con el carrito vacío	No se registra ningún pedido
Completar una compra válida	Se guarda el pedido y se descuenta el stock
Consultar pedidos de otro usuario	No se permite el acceso
Añadir un videojuego duplicado	Se rechaza la inserción
Eliminar un videojuego incluido en pedidos	Se conserva el videojuego y su historial
Seleccionar el tema oscuro	Se aplica en toda la aplicación
Cerrar sesión	Se eliminan los datos de autenticación y carrito

3. Normas generales de programación

Durante todo el desarrollo deberán cumplirse las siguientes normas:

1. Las consultas SQL se implementarán exclusivamente en `includes/funciones_bd.php`.
2. Las funciones generales reutilizables se implementarán en `includes/funciones.php`.
3. Se utilizarán `header.php` y `footer.php` para evitar repetir la estructura HTML.
4. Se utilizará `require_once` para cargar los archivos necesarios.
5. Se empleará `GET` para consultar información y `POST` para realizar modificaciones.
6. Se utilizarán sentencias preparadas y se validarán los datos recibidos.
7. No se guardarán contraseñas en texto plano.
8. Se comprobarán los permisos antes de realizar operaciones restringidas.
9. Se utilizarán sesiones para la autenticación y el carrito, y cookies para las preferencias.
10. El código deberá estar organizado, comentado cuando sea necesario y evitar duplicaciones.

4. Entrega del proyecto

El alumnado deberá entregar:

- La carpeta completa de GameWorld con los archivos PHP terminados.
- Los archivos `includes/funciones.php` e `includes/funciones_bd.php` creados y desarrollados.
- El archivo SQL de la base de datos.
- Un documento breve que incluya las pruebas realizadas, sus resultados y capturas de pantalla.

5. Criterios de comprobación

Para considerar completado el proyecto, la aplicación deberá permitir:

- Registrar e identificar usuarios correctamente.
- Consultar y buscar videojuegos.
- Gestionar un carrito mediante sesiones.
- Registrar pedidos y actualizar el stock.
- Consultar el historial de compras.
- Administrar el catálogo con permisos.
- Guardar preferencias mediante cookies.
- Ejecutar las operaciones de manera segura y sin duplicar innecesariamente código.